INTERVIEW GUIDE

AWS 3-Tier Portfolio Project

Implementation, Architecture, Security, Troubleshooting & Interview Presentation Guide

DOMAIN
chaitanyanaik.site

AWS REGION
ap-south-1 (Mumbai)

STACK
React/Vite • S3 • Route 53 • ALB • ACM • EC2 • Spring Boot • RDS MySQL • IAM/SSM • CloudWatch

EXECUTIVE ARCHITECTURE

Browser → Route 53 → S3 static website (frontend)

Browser → Route 53 → ALB/ACM → private EC2/Spring Boot → private RDS MySQL (API/data)

Important: the current direct S3 website is HTTP. The API is HTTPS. This is a deliberate no-CloudFront trade-off and is documented honestly in this guide.

01

Project Overview

The project is a personal portfolio application deployed as a three-tier AWS design. The presentation tier is a React/Vite static site, the application tier is a Spring Boot REST API, and the data tier is MySQL on Amazon RDS.

The project was intentionally built without Docker, Kubernetes, Terraform, Jenkins or CloudFront. The objective was to learn the AWS fundamentals directly: VPC networking, routing, security groups, DNS, TLS, EC2, RDS, load balancing, IAM, Systems Manager, monitoring and application troubleshooting.

TIER	AWS / TECHNOLOGY	CURRENT ROLE
Presentation	S3 + Route 53	Serves React build
Application ingress	ALB + ACM	HTTPS API endpoint and health checks
Application	Private EC2 + Spring Boot	REST API on port 8080
Data	Private RDS MySQL	Stores contact messages
Operations	IAM + SSM + CloudWatch	Administration and monitoring

The contact form is the proof of end-to-end integration: a browser submission reaches the HTTPS API, is processed by Spring Boot, and is persisted into the RDS table **contact_messages**.

02 · 03

Network Architecture & Security Groups

VPC

portfolio-vpc, CIDR 10.0.0.0/16. The network is divided by trust level.

SUBNET	CIDR	PURPOSE
Public AZ1	10.0.1.0/24	ALB / public resources
Public AZ2	10.0.2.0/24	Second ALB subnet
Private App AZ1	10.0.11.0/24	EC2 backend
Private App AZ2	10.0.12.0/24	App capacity / AZ separation
Private DB AZ1	10.0.21.0/24	RDS subnet group
Private DB AZ2	10.0.22.0/24	RDS subnet group

Routing: the public route table sends 0.0.0.0/0 to the Internet Gateway. Private application routes use NAT gateways for outbound access. Database route tables remain local-only; RDS does not need direct internet access.

SECURITY GROUP	INBOUND	TRUSTED SOURCE
portfolio-alb-sg	TCP 80, 443	Internet
portfolio-ec2-sg	TCP 8080	ALB security group
portfolio-rds-sg	TCP 3306	EC2 security group

This is stronger than opening the backend or database to a broad IP range. EC2 trusts the ALB; RDS trusts EC2. There is **no public SSH rule**.

Why this matters in an interview: the application server and database are not simply placed on the public internet. The architecture creates explicit trust boundaries and forces traffic through controlled entry points.

04

Step-by-Step Build

4.1 APPLICATION PREPARATION

1. Clone the portfolio repository.
2. Build the React frontend with npm run build; production files appear under frontend/dist.
3. Build the Spring Boot backend with ./mvnw clean package -DskipTests.
4. Add a dedicated /health endpoint returning {"status":"UP"} so the ALB can perform a simple health check.

4.2 AWS NETWORKING

1. Create the VPC and six subnets.
2. Attach the Internet Gateway.
3. Create public and private route tables.
4. Create NAT gateways for private application egress.
5. Keep DB subnets private and avoid a default internet route.

4.3 RDS

1. Create MySQL RDS named portfolio-mysql.
2. Database name: portfolio_db.
3. Use a DB subnet group containing the private DB subnets.
4. Set Public access to No.
5. Attach portfolio-rds-sg so port 3306 is reachable only from the application tier.
6. Use cost-conscious single-AZ configuration for the portfolio; this is not the same as production-grade Multi-AZ HA.

Backend Deployment, IAM & SSM

An Amazon Linux 2023 EC2 instance was used for the working backend deployment. Java 17 (Amazon Corretto) and Git were installed. The repository was cloned under /home/ssm-user/sample_portfolio. The backend uses environment variables instead of embedding database credentials in the source:

```
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/portfolio_db?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
server.port=8080
spring.jpa.hibernate.ddl-auto=update
management.endpoints.web.exposure.include=health
```

Runtime values are supplied as DB_HOST, DB_USERNAME and DB_PASSWORD. Never publish the actual password in GitHub, screenshots or documentation. Built with **./mvnw clean package -DskipTests**; started with the packaged JAR. The process was verified with ss -ltnp and the health endpoint with curl.

IAM ROLE

The EC2 instance uses IAM role **portfolio-ec2-role** with AmazonSSMManagedInstanceCore and CloudWatchAgentServerPolicy.

SSM

Allows administrative access without opening SSH port 22 or managing an SSH private key.

CLOUDWATCH AGENT

Provides host-level monitoring/metrics.

LEAST EXPOSURE

EC2 remains private and the public entry point is the ALB.

Load Balancer, TLS, DNS & Frontend

APPLICATION LOAD BALANCER

Named **portfolio-alb**, placed in the two public subnets. Target group **portfolio-backend-tg**, port 8080, health check path /health, success code 200. Listeners: HTTP 80 redirects to HTTPS 443. HTTPS 443 uses the ACM certificate for the domain/wildcard. This gives the API a stable HTTPS endpoint without exposing the EC2 server directly.

RECORD	TARGET	PURPOSE
chaitanyanaik.site	S3 website alias	Frontend
api.chaitanyanaik.site	ALB alias	Backend API

The domain's authoritative nameservers were moved from the registrar to Route 53, after which public DNS resolution worked.

FRONTEND DEPLOYMENT

1. Create S3 bucket chaitanyanaik.site in Mumbai.
2. Upload the contents of frontend/dist.
3. Enable Static Website Hosting with index.html.
4. Allow public object reads because the S3 website endpoint must serve the public site.
5. Create the Route 53 apex A Alias targeting the S3 website endpoint.
6. Keep api.chaitanyanaik.site pointing to the ALB.

```
VITE_API_URL=https://api.chaitanyanaik.site
```

The contact form uses the API base URL rather than a relative /api path so the S3-hosted frontend can call the separate backend domain.

CORS Troubleshooting & Database Verification

THE PROBLEM

The first production contact submission failed. The browser console reported that the preflight request did not receive an Access-Control-Allow-Origin header for http://chaitanyanaik.site.

ROOT CAUSE

The backend CORS configuration contained development origins and an old placeholder production origin, but not the real production website origin.

FIX

Add the production origin to Spring's allowedOrigins list, rebuild the backend, restart the JAR, and retest.

An excellent interview example: the difference between network reachability and browser security policy. The API was reachable, but the browser correctly blocked the cross-origin request.

Because RDS is private, verification was performed from the private EC2 environment using Session Manager. The database contains the table **contact_messages**.

```
mysql -h "$DB_HOST" -u "$DB_USERNAME" -p portfolio_db
SHOW TABLES;
SELECT * FROM contact_messages;
SELECT * FROM contact_messages ORDER BY id DESC;
```

A real contact form submission was confirmed in the table. This proves the complete path:

browser → S3 frontend → HTTPS API → ALB → EC2/Spring Boot → RDS MySQL

11

Security Assessment

11.1 STRONG CONTROLS IMPLEMENTED

- RDS Public access is disabled.
- RDS resides in private DB subnets.
- EC2 resides in private application subnets.
- EC2 port 8080 is trusted only from the ALB security group.
- RDS port 3306 is trusted only from the EC2 security group.
- No inbound SSH is required; SSM is used instead.
- The API uses HTTPS with ACM and ALB.
- Health checks prevent the ALB from routing to unhealthy backend targets.
- Database credentials are supplied through environment variables rather than source code.
- CloudWatch Agent is installed/configured.
- RDS encryption and backups were enabled within the project's cost constraints.

11.2 KNOWN LIMITATIONS — BE HONEST IN INTERVIEWS

- **Frontend HTTP:** direct S3 website endpoints are HTTP-only. The initial page load is not protected by TLS.
- **Public S3 website:** public object reads are required by this hosting model.
- **No WAF:** the project does not currently use AWS WAF.
- **Secrets:** environment variables are better than hard-coding, but Secrets Manager/Parameter Store would be stronger.
- **RDS HA:** single-AZ is a budget decision, not production-grade high availability.
- **Schema management:** Hibernate ddl-auto=update is convenient for a portfolio; Flyway/Liquibase is better for controlled production migrations.
- **ASG:** configured during the project, but the final working backend was proven on the standalone EC2. Do not claim automatic replacement is production-ready until the launch template fully bootstraps the application and has been tested.

Security conclusion: a good learning/portfolio architecture with meaningful network isolation, but it should not be marketed as fully production-hardened. The most important improvement is an HTTPS-capable frontend architecture.

Why We Chose Each Service

SERVICE	WHY CHOSEN	TRADE-OFF
S3	Simple, cheap static hosting	Website endpoint is HTTP
Route 53	AWS-native DNS and aliases	Does not terminate TLS
ALB	Stable API entry, HTTPS, health checks	Has ongoing cost
EC2	Control over Java/Spring runtime	OS/process management required
RDS	Managed MySQL	Costs more than self-hosting
Private subnets	Reduce exposure	NAT is needed for private egress
SSM	No SSH exposure	Requires IAM/SSM setup
CloudWatch	Operational visibility	Needs dashboards/alarms for mature monitoring
No CloudFront	Matches project requirement and keeps architecture simple	No HTTPS for direct S3 website

Cost strategy: the project prioritizes understanding core AWS building blocks while staying cost-conscious. A production redesign would revisit NAT gateway count, RDS HA, TLS/CDN, secrets, WAF and automation based on actual requirements and budget.

Troubleshooting Lessons

PROBLEM	DIAGNOSIS	SOLUTION
ALB target unhealthy	Backend/health endpoint not ready	Added /health and used it as target health check
CORS failure	Production origin missing	Added http://chaitanyanaik.site to allowedOrigins
Backend failed after restart	DB_* variables missing in new shell	Export runtime DB variables before starting JAR
Route 53 S3 error	Endpoint entered as a value instead of selecting Alias target	Use Alias → S3 website endpoint
Private DB verification	RDS has no public access	Use SSM on EC2 and run MySQL client

Interview Presentation — 90 Seconds

> “I built and deployed a three-tier portfolio application on AWS. The frontend is a React/Vite static build hosted on S3 and exposed through Route 53. The backend is a Spring Boot application running on a private EC2 instance behind an internet-facing Application Load Balancer. The ALB terminates HTTPS using ACM and forwards requests to port 8080. The backend connects to a private RDS MySQL database, and the database security group only accepts traffic from the application security group. I use Systems Manager instead of public SSH and CloudWatch for monitoring. I intentionally did not use CloudFront, so the direct S3 website remains HTTP; the API is HTTPS. I can explain that as a known trade-off and describe the production upgrade path. I also troubleshot real CORS, process, health-check and database connectivity issues during deployment.”

Common Interview Questions

WHY IS EC2 PRIVATE?

The ALB is the public ingress. EC2 does not need direct internet exposure.

WHY IS RDS PRIVATE?

Only the backend needs database access; exposing MySQL publicly would increase attack surface.

WHY USE AN ALB?

HTTPS termination, stable endpoint, health checks and future horizontal scaling.

WHY NO SSH?

SSM Session Manager provides controlled management access without opening port 22.

WHY IS THE FRONTEND HTTP?

Direct S3 website endpoints do not support HTTPS. A separate TLS-capable frontend service would be required.

HOW DID YOU PROVE THE DATABASE WORKED?

I connected through the private EC2 using Session Manager and queried contact_messages after submitting the form.

WHAT WOULD YOU IMPROVE?

HTTPS frontend, Secrets Manager, WAF/rate limiting, CloudWatch alarms, migration tooling, Multi-AZ RDS and fully automated ASG bootstrapping.

Final Checklist & Production Roadmap

16. FINAL CHECKLIST BEFORE PUBLISHING

- Verify no database password or AWS credentials are present in Git history.
- Rotate any credential that may have been exposed during development.
- Do not include passwords in screenshots.
- Verify the public API does not expose submitted contact messages through an unauthenticated GET endpoint.
- Upload the PDF documentation to the GitHub repository.
- Add a concise README describing the architecture and known HTTPS limitation.
- Use the LinkedIn draft included with this deliverable.
- In interviews, say exactly what is implemented today and clearly label future improvements.

18. PRODUCTION ROADMAP

1. Move the frontend to an HTTPS-capable architecture if HTTPS is required.
2. Use Secrets Manager or Parameter Store.
3. Add WAF/rate limiting and alarms.
4. Automate EC2 bootstrap/AMI so ASG replacement instances are healthy automatically.
5. Use Flyway/Liquibase migrations.
6. Consider Multi-AZ RDS when availability requirements justify the cost.
7. Add CI/CD after the manual deployment process is fully understood.
8. Protect or remove any endpoint that could expose private contact submissions.

Resource Inventory

RESOURCE	VALUE
VPC	portfolio-vpc / 10.0.0.0/16
IGW	portfolio-igw
ALB	portfolio-alb
Target group	portfolio-backend-tg / :8080 / /health
RDS	portfolio-mysql / MySQL / private
Database	portfolio_db
RDS table	contact_messages

RESOURCE	VALUE
EC2 SG	portfolio-ec2-sg
RDS SG	portfolio-rds-sg
IAM role	portfolio-ec2-role
Launch template	portfolio-backend-template
ASG	portfolio-backend-asg (bootstrap needs verification before claiming active HA)
S3 bucket	chaitanyanaik.site
API	api.chaitanyanaik.site
Region	ap-south-1 (Mumbai)

Prepared from the actual implementation and troubleshooting performed for this project. It distinguishes the current working deployment from future production improvements.